

Product Requirements Document (PRD)

CPSC 3750 – Web Application Development | Clemson University

Project Overview

Project Name:

Multiplayer Battleship System

Team Members:

Shaun Whitt

Jack Stivers

One-Sentence Description:

A server-managed multiplayer Battleship system that tracks players, enforces game rules, and maintains persistent game state.

Problem Statement

Turn-based multiplayer games require a reliable system to manage players, enforce rules, and keep track of game progress. Without a centralized system, players must manually coordinate turns, validate moves, and track hits or eliminations, which can easily lead to mistakes or inconsistent gameplay.

This problem affects players who want to participate in structured multiplayer games where rules are consistently enforced. Without a system managing the game, players may take turns incorrectly, make invalid moves, or lose track of game progress.

A server-managed system solves this problem by ensuring that all moves are validated, turns are enforced correctly, and the overall game state remains consistent for all players.

Target Users**Primary user:**

Players participating in a multiplayer Battleship game.

Secondary user:

Developers or testers interacting with the system while verifying gameplay functionality.

User skill level:

Basic computer familiarity; no technical expertise required for players.

Environment:

Desktop computers interacting with the game server through a client application or testing interface.

Goals & Success Criteria

Functional Success

The system should:

Allow players to join a game session

Manage turn rotation between players

Validate moves according to Battleship rules

Track hits, misses, and ship destruction

Eliminate players when all of their ships are sunk

Determine and announce the winner of the game

Persistence Requirements

Game information should remain stored so that:

Active games are not lost if the server restarts

Player statistics and game results remain available

Performance Expectations

The system should process moves quickly enough that gameplay feels responsive and natural.

Scalability Expectations

The system should support multiple games running at the same time without interference between them.

Correctness Validation

The system will be considered successful if:

Game rules are consistently enforced

Turns occur in the correct order

Invalid moves are rejected

Game results are recorded correctly

Core Features (Behavioral Description Only)

Game Creation

The system allows a new Battleship game to be created where players can join before gameplay begins.

Player Participation

Players can join an available game and become active participants.

Turn Enforcement

The system ensures that only the correct player can make a move during their turn.

Move Processing

Players can target a location on another player's board, and the system determines whether the move results in a hit or miss.

Game Progress Tracking

The system tracks ship damage, player eliminations, and overall game progress.

Game Completion

Once all but one player has been eliminated, the system declares the remaining player the winner.

Statistics Tracking

Basic statistics such as games played, wins, and losses are recorded.

User Stories

User Story 1

As a player,
I want to join a Battleship game,
so that I can participate in multiplayer gameplay.

User Story 2

As a player,
I want the system to enforce turn order,
so that gameplay remains fair and organized.

User Story 3

As a player,
I want my moves to be validated by the system,
so that illegal or invalid moves cannot occur.

User Story 4

As a player,
I want the system to track hits and eliminations,
so that the game progresses correctly and ends with a clear winner.

Constraints

Technology Constraints

The system must follow a persistent multiplayer client/server system.

Hosting Constraints

The application must use a server and database, a multiplayer API, and a relational schema.

Persistence Requirements

Game data and player statistics must be stored so that they remain available even if the server restarts.

Security Requirements

The system must prevent players from making moves when it is not their turn and must reject invalid or malformed actions.

Deadline Constraints

Phase 1: March 31

Phase 2: April 17

Phase 3: April 21

Non-Goals (Out of Scope)

The following features are intentionally not included:

Advanced graphical game interfaces

Mobile applications

Chat or messaging systems between players

Player matchmaking systems

Advanced AI gameplay strategies

Full user account systems

Risks & Assumptions

Technical Risks

Turn management logic could lead to errors if not handled carefully.

Game state persistence could cause data inconsistencies if updates are not handled properly.

Multplayer synchronization issues may occur if multiple actions happen at the same time.

Team Risks

Uneven workload distribution between team members.

Communication challenges during development.

Dependency Risks

Limitations of the development environment or required frameworks.

Potential issues with external tools or libraries.

Assumptions

Players follow normal gameplay behavior.

The server environment remains stable during development and testing.

Required development tools remain available throughout the project timeline.